

Script trình bày — Self-RAG-Inspired Legal QA

Đồ án cuối kỳ CS2308.CH203 · Vũ Đức Mạnh · Nguyễn Hoàng Nam

Script cũng đã được nhúng sẵn vào phần Speaker Notes của file .pptx — mở Presenter View trong PowerPoint để xem trực tiếp khi trình bày.

Slide 1 — Trang bìa

Kính chào Thầy/Cô. Em xin phép trình bày đồ án cuối kỳ môn Chuyên đề nghiên cứu và ứng dụng NLP, đề tài "Self-RAG-Inspired Retrieval-Augmented Generation cho hỏi–đáp pháp luật tiếng Việt", lấy cảm hứng từ bài báo Self-RAG của Asai và cộng sự, ICLR 2024. Nhóm em gồm hai thành viên: Vũ Đức Mạnh và Nguyễn Hoàng Nam, sinh viên Trường Đại học Công nghệ Thông tin, ĐHQG-HCM.

Slide 2 — Agenda

Trong phần trình bày hôm nay, em sẽ đi qua sáu phần: động lực nghiên cứu; hành trình từ RAG đến Self-RAG; bài toán và kiến trúc của đồ án; dữ liệu và triển khai; thực nghiệm và kết quả; và cuối cùng là thảo luận, demo trực tiếp và kết luận.

Slide 3 — Động lực: LLM Hallucination & vai trò của RAG

Vấn đề đầu tiên là hallucination — LLM sinh thông tin sai sự thật nhưng trình bày rất tự tin, đặc biệt nguy hiểm trong lĩnh vực đòi hỏi độ chính xác cao như pháp luật, vì một trích dẫn sai có thể gây hậu quả thực tế. Retrieval-Augmented Generation, do Lewis và cộng sự đề xuất năm 2020, giải quyết vấn đề này bằng cách điều kiện hoá việc sinh văn bản theo các đoạn truy xuất từ kho ngữ liệu ngoài. Pipeline cơ bản: câu hỏi qua retriever lấy đoạn liên quan, rồi đưa vào generator để sinh câu trả lời.

Slide 4 — Hạn chế của RAG truyền thống & giới thiệu Self-RAG (paper gốc)

RAG truyền thống vẫn còn hạn chế: retrieve với chiến lược cố định bất kể câu hỏi có cần hay không, đưa toàn bộ đoạn truy xuất cho generator vô điều kiện, và không tự kiểm tra được câu trả lời có được hỗ trợ hay hữu ích hay không. Đây là động lực ra đời của Self-RAG, Asai và cộng sự, ICLR 2024: huấn luyện một LM tự sinh reflection token xen kẽ văn bản, gồm bốn loại – RETRIEVE quyết định có cần truy xuất, ISREL đánh giá đoạn có liên quan, ISSUP đánh giá câu trả lời có được hỗ trợ, ISUSE đánh giá câu trả lời có hữu ích. Điểm then chốt: bản gốc đòi hỏi fine-tune cả critic model lẫn generator để sinh ra các token này.

Slide 5 – Vấn đề nghiên cứu & đóng góp

Câu hỏi nghiên cứu của đề án: trên miền văn bản pháp luật tiếng Việt, thêm cơ chế tự phản tư vào RAG có cải thiện độ chính xác truy xuất và chất lượng câu trả lời hay không, khi các tín hiệu phản tư được mô phỏng bằng prompting thay vì fine-tune? Ràng buộc thực tế: chỉ có Colab free, không đủ ngân sách để fine-tune như paper gốc, nên nhóm mô phỏng cả bốn tín hiệu phản tư hoàn toàn bằng prompting JSON trên một LLM cố định qua Groq API. Bốn đóng góp chính: tái tạo bốn tín hiệu phản tư chỉ bằng prompting; so sánh định lượng trên corpus pháp luật tiếng Việt thật;

ablation tái hiện được để tách bạch tác động của ISREL; và trình bày trung thực các thất bại, trở ngại kỹ thuật khi vận hành LLM-judge quy mô lớn.

Slide 6 — Kiến trúc ba hệ thống so sánh

Nhóm xây dựng ba hệ thống để so sánh. Hệ 1, No-RAG: câu hỏi đi thẳng qua LLM ra câu trả lời — baseline dưới. Hệ 2, Standard RAG: retrieve cố định top-5 rồi đưa cho LLM cùng câu hỏi — baseline giữa. Hệ 3, Self-RAG-inspired, là hệ chính của đồ án: câu hỏi qua RETRIEVE quyết định có cần truy xuất, retrieve top-k, qua ISREL lọc đoạn liên quan, generate câu trả lời kèm trích dẫn, qua ISSUP và ISUSE tự đánh giá, rồi trả về câu trả lời kèm reflection report đầy đủ.

Slide 7 — Self-RAG-inspired: thiết kế 4 module phản tư

Mỗi module là một lượt gọi LLM riêng biệt yêu cầu đúng một JSON object — không module nào là model được huấn luyện riêng. RETRIEVE trả Yes/No/Continue, mặc định RETRIEVE khi không parse được để thiên về an toàn. ISREL đánh giá theo lô tất cả đoạn trong một lượt gọi, lỗi thì mặc định

RELEVANT. Generate chọn một trong hai prompt thích ứng tùy còn evidence hay không. ISSUP chỉ gọi khi còn ít nhất một đoạn liên quan, nếu không thì gán thẳng NOT_APPLICABLE. ISUSE luôn được gọi vì hữu ích là thuộc tính của câu trả lời cuối, không phụ thuộc evidence.

Slide 8 — Dataset: VLSP2025 DRiLL

Nhóm dùng bộ VLSP2025 DRiLL — benchmark pháp luật tiếng Việt thật, gồm 2.157 văn bản, 59.636 điều luật, và 2.190 câu hỏi có nhãn thật trong train.json. Vì hạn nộp shared-task đã qua trước khi đề án bắt đầu, tập public test và private test có nhãn ẩn không dùng được để đánh giá nên nhóm loại khỏi phạm vi đề án. Mọi so sánh trong báo cáo dùng chung một dev set cố định 250 câu lấy từ train.json với seed 42, đảm bảo ba hệ thống được so trên đúng cùng một tập câu hỏi.

Slide 9 — Retriever & Generator

Infrastructure

Retriever hoạt động theo bốn bước: chunking ưu tiên theo đoạn văn, dự phòng cửa sổ 800 ký tự overlap 150; embedding bằng bkai-foundation-models/vietnamese-bi-encoder; index bằng FAISS

IndexFlatIP; và truy xuất, chấm điểm luôn ở cấp độ điều luật để khớp độ chi tiết nhãn vàng. Generator: toàn bộ generate và bốn module phản tư dùng chung một LLM nền qua Groq API, model không hardcode mà tự dò danh sách khả dụng và tự chuyển khi hết quota. Mỗi prompt yêu cầu đúng một JSON object, parser chịu lỗi, tự bóc code-fence hoặc dùng regex dự phòng. Có checkpoint theo từng dòng để an toàn khi mất kết nối Colab.

Slide 10 – Thách thức kỹ thuật khi triển khai

Ba lỗi kỹ thuật đã xử lý. Một, model "thinking" luôn sinh khối think trước JSON khiến parse sai, âm thầm rơi về nhãn mặc định — sửa bằng cách strip khối này trước khi parse và lưu. Hai, khi mọi model cùng hết quota, vòng lặp cũ vẫn tiếp tục ghi bản ghi mặc định — sửa bằng cách kiểm tra model khả dụng trước mỗi câu và dừng sớm kèm log rõ ràng. Ba, một prompt judge-support ghép nhiều điều luật dài vượt giới hạn token mỗi phút, bị hiểu nhầm là hết quota và khoá nhầm cả ba model — sửa bằng cách không đánh dấu model hỏng khi gặp lỗi 413, đồng thời giới hạn độ dài mỗi điều luật.

Slide 11 – Thiết lập thực nghiệm & kiểm chứng chất lượng dữ liệu

Cỡ mẫu thực tế đi qua một funnel: từ 250 câu dev set, Self-RAG hoàn thành 202 câu (80,8%), sau khi loại câu bị nhiễu còn 187 câu được chấm đáng tin cậy (92,6%). Đây là giới hạn cứng của free-tier Groq, không phải lỗi phương pháp luận — vì lý do sử dụng API key free nên kết quả với bộ data so sánh bị hạn chế. Metrics: truy xuất dùng Recall, Precision, MRR bằng Python thuần; chất lượng câu trả lời dùng LLM judge cùng prompt, temperature 0, trên ba trục Correctness, Support, Usefulness. Kiểm chứng chất lượng judge trên 187 câu: nhiệm think chỉ còn 3,2%, tỷ lệ toàn bộ judge cùng rơi mặc định giảm còn 6,4% so với 99% trước khi sửa lỗi, và phân bố nhãn đa dạng thật ở cả ba hệ — nên dữ liệu lần pull này dùng được cho báo cáo, không cần chạy lại.

Slide 12 – Kết quả chính: so sánh 3 hệ thống

Đây là kết quả chính trên 187 câu hỏi. No-RAG chỉ đạt 16% Correctness và 50,8% Usefulness, Support không áp dụng vì không dùng evidence. Standard RAG đạt 25,7% Correctness, 53,5% Support, nhưng chỉ 25,7% Usefulness. Self-RAG-inspired vượt trội trên cả ba chỉ

số cùng lúc: 33,2% Correctness, 62% Support, 62,6% Usefulness – bằng chứng mạnh nhất cho giá trị của cơ chế phản tư, thắng cả hai baseline trên mọi chỉ số chất lượng câu trả lời cùng lúc.

Slide 13 – Retrieval Quality: trước/sau ISREL

Trước khi qua ISREL, kết quả gần giống hệ Standard RAG, đúng như kỳ vọng vì dùng chung retriever, Precision chỉ 0,129. Sau ISREL, Precision tăng lên 0,28 – tăng khoảng 2,2 lần – đổi lại Recall giảm tương đối 15 đến 17%, còn 0,425. Đây là đánh đổi Precision-Recall hợp lý, và hành vi này ổn định, tái hiện nhất quán qua ba lần đánh giá độc lập của đồ án.

Slide 14 – Ablation: ISREL → ISSUP (phát hiện quan trọng nhất)

Đây là phát hiện quan trọng nhất của đồ án: ablation kiểm tra ISREL có thực sự cải thiện tính có-căn-cứ hay không. Với câu hỏi mà ISREL không lọc bớt gì, tỷ lệ FULLY_SUPPORTED chỉ 39%. Với câu hỏi mà ISREL lọc bớt ít nhất một đoạn nhiễu, tỷ lệ tăng lên 90,3% – tăng hơn gấp đôi, 2,3 lần. Trung bình mỗi câu retrieve 5 điều luật, sau lọc còn 2,39, giữ lại khoảng 52%; và

78,8% câu hỏi có RETRIEVE bằng Yes bị lọc bớt ít nhất một đoạn. Kết quả tái hiện nhất quán qua ba lần đánh giá độc lập.

Slide 15 – Phân bố Reflection Signals

Phân bố tín hiệu phản tư trên 202 câu: RETRIEVE – 95,5% cần truy xuất, chỉ 4,5% không cần, phần lớn là câu hỏi chính sách hoặc kiến thức chung. ISSUP – 63,4% Fully supported, 17,3% Partially, chỉ 1% No support, 18,3% Not Applicable. ISUSE – 61,9% Useful, 34,2% Partially useful, chỉ 4% Not useful.

Slide 16 – Case Study: minh chứng thành công

Hai case study thành công. Với câu hỏi con dâu có được thừa kế khi bố mẹ chồng mất không di chúc, Standard RAG trả lời sai vì bị đánh giá không đủ căn cứ, trong khi Self-RAG-inspired trả lời đúng, có trích dẫn cụ thể quyền thừa kế thuộc về vợ chồng, con cái, cha mẹ. Với câu hỏi số tài khoản chứng khoán công ty quản lý quỹ nước ngoài được mở, trong 5 điều truy xuất chỉ 1 điều liên quan – Standard RAG dùng cả 5 điều thô nên bị nhiễu trả lời sai, còn Self-RAG-inspired

nhờ ISREL lọc còn đúng 1 điều nên trả lời chính xác,
Fully Supported.

Slide 17 – Case Study: hạn chế thực tế

Hai kiểu thất bại còn tồn tại. Một, hallucination dù trích dẫn đúng định dạng: câu hỏi về cơ quan cho thuê đất xây đại sứ quán, Self-RAG trả lời sai là Bộ Xây dựng thay vì UBND cấp tỉnh, dù format trích dẫn hợp lệ – bài học: ISSUP chỉ kiểm tra có trích dẫn, không đảm bảo hiểu đúng nội dung. Hai, lỗi tăng truy xuất mà phản tư không cứu được: câu hỏi về trách nhiệm hình sự khi mua bán tài khoản ngân hàng, điều luật đúng chưa từng nằm trong top-5 truy xuất – bài học: tự phê bình chỉ đánh giá được evidence đã có, không thể khôi phục evidence chưa từng truy xuất.

Slide 18 – Discussion: nghịch lý Usefulness của Standard RAG

Nghịch lý Usefulness của Standard RAG: nhận đúng 5 điều cố định dù liên quan hay không, nên khi không đủ liên quan nó trung thực từ chối trả lời – đúng nguyên tắc nhưng bị chấm không hữu ích, chỉ 25,7%, thấp hơn cả No-RAG 50,8%. Self-RAG-inspired giải quyết được

đánh đổi này: ISREL lọc nhiều trước khi generate nên khi trả lời, hệ thống tự tin và trúng đích hơn – vừa đúng hơn 33,2% so với 25,7%, vừa hữu ích hơn 62,6% so với 25,7%, cùng lúc. Đây là bằng chứng định lượng trực tiếp cho giá trị của self-reflection.

Slide 19 – Demo hệ thống

Em xin phép demo trực tiếp hệ thống, chạy trên Colab ở notebook 03, mục 11. Quy trình sáu bước: nhập câu hỏi, RETRIEVE quyết định truy xuất, retrieve top-k và ISREL lọc đoạn liên quan, generate câu trả lời kèm trích dẫn, ISSUP và ISUSE tự đánh giá, trả về câu trả lời kèm reflection report. Ví dụ minh họa: câu hỏi về số tài khoản chứng khoán công ty quản lý quỹ nước ngoài – RETRIEVE Yes, truy xuất 5 điều, ISREL giữ đúng 1 điều liên quan, câu trả lời chính xác kèm trích dẫn, đánh giá Fully Supported, Useful.

Slide 20 – Kết luận & hướng phát triển

Self-reflection vẫn có giá trị ngay cả khi chỉ mô phỏng bằng prompting. Self-RAG-inspired vượt trội cả hai baseline trên mọi chỉ số chất lượng cùng lúc; ISREL giúp tăng hơn gấp đôi tỷ lệ câu trả lời được hỗ trợ đầy

đủ, tái hiện qua ba lần đánh giá độc lập; và đề án giải quyết được đánh đổi đúng-với-hữu ích mà baseline truy xuất cố định gặp phải. Ý tưởng cốt lõi của Self-RAG vẫn có giá trị ở dạng hoàn toàn dựa trên prompting, không tinh chỉnh, trên ngôn ngữ ít tài nguyên và lĩnh vực chuyên biệt. Hướng phát triển: tinh chỉnh một critic model tiếng Việt nhẹ thay cho judge dựa trên prompting, đánh giá trên mẫu lớn hơn, và khám phá phản tư ở cấp độ token ngay trong lúc sinh văn bản.

Slide 21 – Limitations & lời cảm ơn / Q&A

Nhóm xin nêu thẳng thắn các hạn chế. Tín hiệu phản tư mô phỏng bằng prompting trên LLM tổng quát, không phải critic huấn luyện chuyên biệt như paper gốc, có thể kém hiệu chỉnh hơn. Dựa vào LLM judge thay vì người gán nhãn, khoảng 6,4% số dòng vẫn có dấu hiệu nhiễu dù đã sửa lỗi. Vì lý do sử dụng API key free nên kết quả với bộ data so sánh bị hạn chế, chỉ 187 trên 250 câu hỏi dự kiến — giới hạn cứng do rate-limit, không phải lựa chọn phương pháp luận. Đề án chỉ đánh giá trên một lĩnh vực, một ngôn ngữ, nên khả năng khái quát hoá chưa được kiểm chứng. Và một số lỗi xuất phát hoàn toàn từ tầng truy xuất mà không module phản tư nào phía sau sửa được. Nhóm em xin

cảm ơn Thầy/Cô đã lắng nghe, rất mong nhận được
góp ý ạ.
